



POLITECNICO DI MILANO

A SIMPLE MODEL FOR VIRUS SPREADING  
MIDDLEWARE TECHNOLOGIES FOR DISTRIBUTED  
SYSTEMS

---

## Document Design

---

*Authors:*

Livio BIONDO  
Francesco GIANCHINO  
Giulia VALCAMONICA

*Professor:*

Luca MOTTOLA

2020-2021

## Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>List of Figures</b>                                      | <b>I</b> |
| <b>1 Introduction</b>                                       | <b>1</b> |
| 1.1 Purpose . . . . .                                       | 1        |
| 1.2 Scope . . . . .                                         | 1        |
| <b>2 Overall Description</b>                                | <b>2</b> |
| 2.1 External Interface Requirements . . . . .               | 2        |
| 2.1.1 User Interfaces . . . . .                             | 2        |
| 2.1.2 Requirements . . . . .                                | 2        |
| 2.2 Implementation . . . . .                                | 3        |
| 2.2.1 Component dependency and implementation order . . . . | 3        |
| 2.2.2 Implementation choices . . . . .                      | 4        |

## List of Figures

|                                                |   |
|------------------------------------------------|---|
| 1 Diagram Of The Individuals' States . . . . . | 4 |
|------------------------------------------------|---|

# 1 Introduction

## 1.1 Purpose

The aim of this document is to describe what the software was intended for and highlight the functionalities required to fulfill all the client's requests. It also provides a technical and detailed description of the architectural and design choices that have been made during the development of the software along with the list of the main components of the system and the interfaces between them.

A Simple Model For Virus Spreading is a software that is intended to help the study about how a virus spreads over time.

## 1.2 Scope

A Simple Model For Virus Spreading is a software intended to be used for the sole purpose of managing information about virus spreading.

In order to do fulfill its scope the program will allow users to view at the end of the day the number of healthy, sick and prone to be sick individuals and the countries in which they're placed.

## 2 Overall Description

### 2.1 External Interface Requirements

#### 2.1.1 User Interfaces

Using the interfaces of the system the users can:

- Insert the number of countries that need to be checked.
- Insert the number of the healthy people to be managed by the software and the number of the infected ones.
- Insert the dimension of the map that represent the world to be considered.
- Insert the moving speed of the inserted people.
- Insert the maximum distance at which a virus can spread from an infected subject.
- Insert the interval of time required between updates of the inserted people state.

#### 2.1.2 Requirements

The following requirements are derived in order to fulfill the specified goal.

**Visualize spreading:**

Users view the virus spreading at the end of the day The system asks to the users to insert:

- R1** The number of the healthy individual
- R2** The number of the sick individual
- R3** The length of the world map
- R4** The moving speed of the individuals
- R5** The maximum distance at which the virus can spread
- R6** The time step

For the implementation of the system a strict and methodical approach has to be followed. The two alternatives that have been taken in consideration are the waterfall and the agile models, but considering the detailed specifications and requirements that are already defined for the system the waterfall method is ideal. Following this life cycle involves a sequential execution of the main steps from the collection of information to the deployment of the software. Given these statements it would be easier and faster to arrive to the final product result.

## 2.2 Implementation

The implementation starts after the analysis of two phases. The first step is to divide the system into different areas corresponding its macro components. The second phase is to define the flow of development of the modules based on their dependencies.

### 2.2.1 Component dependency and implementation order

The most critical part is the integration among the macro components, which has to be done one step at the time to identify, in case of problems, the causes of errors or malfunctions.

We decided to start by building first the countries and then the world, successively the method to build the world and to add the countries were made. Then we started to create the first individuals and to add them on the map, each country has a list of the individuals in it and the world has a list of all the countries in the world.

Once that everything was done and successfully tested, the method to calculate the distance among individuals was made along with the methods used to change the status of the individuals and the methods to move the individuals inside and outside their countries.

Then we added the implementation of the methods that grant the view of the virus spreading information at the end of each day.

At this moment the majority of the work was made and we started modifying the program in order to insert the MPI architecture.

### 2.2.2 Implementation choices

As requested by the client in order to implement the software were used c programming language combined with MPI.

In order to simplify our work we've assumed that all the individuals will have the same moving speed and all the country will have the same dimension. Individuals can also change their country during the execution of the program.

#### individuals' movements

The movement of each individual are chosen in a random way among different possible movement direction that have been indicated inside the movement.h file, each time a individual picks a movement we check if the movement will cause the change of the country or the exit from the world map, in the last case by design we try to check if also the opposite direction of the movement can cause to exit the map if not we apply this direction instead of the first one otherwise for this turn the individual will not move.

#### individuals' state

The state of each individual is periodically checked and updated (accordingly to the users' input on the interval of time), if an healthy individual, after a movement, is near an infected one we update inside the individual a counter that is needed to check for how many minutes the individual was near an infected, we also change the status from healthy to infection in progress. When an infection is in progress we could have two cases if the counter is less than ten minutes and the near individuals are not infected then the individual can turn back to the healthy status and the counter is set to zero, if instead the counter is greater (or equal) than ten minutes the infection has occurred and the individual change the status to the infected one. In the infected status the counter is used in order to check if the individual has been infected for ten days, once that the timeline is surpassed the individual can change status to the immune one. An immune individual cannot be infected for three months since acquiring the status, once that this time has been depleted the individual acquires the healthy status.

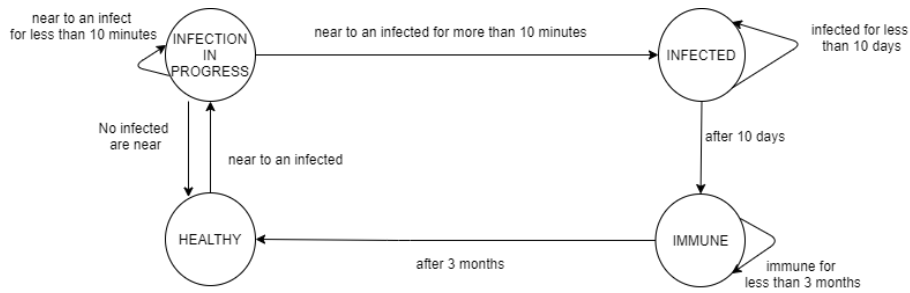


Figure 1: Diagram Of The Individuals' States